

CMP1203 – Fundamentos de Programação Orientada a Objetos

Prof. Me Alexandre Cláudio de Almeida



**PUC
GOIÁS**

Pilares da P00



Três Pilares

- Encapsulamento
- Herança
- Polimorfismo



Software à prova do Futuro

- Se adapta aos requisitos futuros sem alteração
- Permite que você faça alterações e adicione novos recursos facilmente
- Um software de sucesso NÃO é estático

Polimorfismo

- Sem o ENCAPSULAMENTO e a HERANÇA, não é possível o POLIMORFISMO
- Sem o POLIMORFISMO, a POO não seria EFICAZ

O que é o Polimorfismo?

- Significa MUITAS FORMAS.
- Permite que um único nome de método represente um código diferente, selecionado por algum mecanismo automático
- Um NOME pode assumir muitas FORMAS e representar muitos COMPORTAMENTOS diferentes

```
1 //POLIMORFISMO COM HERANÇA
2 // Classe base
3 class Animal {
4     public void fazerSom() {
5         System.out.println("Som de animal");
6     }
7 }
8
9 // Classes derivadas
10 class Cachorro extends Animal {
11     public void fazerSom() {
12         System.out.println("AU AU AU");
13     }
14 }
15
16 class Gato extends Animal {
17     public void fazerSom() {
18         System.out.println("MIAU MIAU");
19     }
20 }
21
22 public class TesteAnimal {
23     public static void main(String[] args) {
24         Animal cachorro = new Cachorro();
25         Animal gato = new Gato();
26
27         cachorro.fazerSom(); // Chama o latido
28         gato.fazerSom(); // Chama o miado
29     }
30 }
```

Abrir




```
1 class ObjetoPersonalidade {
2     public String falar() {
3         return "Eu sou um Objeto";
4     }
5 }
6
7 class ObjetoPessimista extends ObjetoPersonalidade {
8     public String falar() {
9         return "O copo esta meio vazio";
10    }
11 }
12
13 class ObjetoOtimista extends ObjetoPersonalidade {
14     public String falar() {
15         return "O copo esta meio cheio";
16    }
17 }
18
19 class ObjetoIntrovertido extends ObjetoPersonalidade {
20     public String falar() {
21         return "Oi...";
22    }
23 }
24
25 class ObjetoExtrovertido extends ObjetoPersonalidade {
26     public String falar() {
27         return "E ai, bla, bla, bla, voce sabia...";
28    }
29 }
```

```

31 public class TestaPersonalidade {
32     public static void main(String[] args) {
33         ObjetoPersonalidade personalidade = new ObjetoPersonalidade
            ();
34         ObjetoPessimista pessimista = new ObjetoPessimista();
35         ObjetoOtimista otimista = new ObjetoOtimista();
36         ObjetoIntrovertido introvertido = new ObjetoIntrovertido();
37         ObjetoExtrovertido extrovertido = new ObjetoExtrovertido();
38         //capacidade de substituicao permite o seguinte:
39         ObjetoPersonalidade[] personalidades = new
            ObjetoPersonalidade[5];
40         personalidades[0] = personalidade;
41         personalidades[1] = pessimista;
42         personalidades[2] = otimista;
43         personalidades[3] = introvertido;
44         personalidades[4] = extrovertido;
45         //o polimorfismo faz com que a Personalidade pareca ter
            muitos comportamentos diferentes
46         for (int i = 0; i < 5; i++) {
47             System.out.println("\n Personalidade[" + (i + 1) + "]
                diz: " + personalidades[i].falar());
48         }
49     }
50 }

```

Personalidade[1] diz: Eu sou um Objeto

Personalidade[2] diz: O copo esta meio vazio

Personalidade[3] diz: O copo esta meio cheio

Personalidade[4] diz: Oi...

Personalidade[5] diz: E ai, bla, bla, bla, voce sabia...



```
1 class ObjetoPersonalidade {  
2     public String falar() {  
3         return "Eu sou um Objeto";  
4     }  
5  
6     public void mandeFalar(ObjetoPersonalidade objeto) {  
7         System.out.println(objeto.falar());  
8     }  
9 }
```

```
for (int i = 0; i < 5; i++) {  
    System.out.println("\n Personalidade[" + (i + 1) + "] diz: ");  
    personalidade.mandeFalar(personalidades[i]);  
}
```

Polimorfismo

- Adicionar novas funcionalidades a qualquer momento
- Adicionar novas classes que tenham funcionalidades jamais imaginadas
- Sem mudar o código
- Software à Prova do Futuro

Pessoa

```
1 public class Pessoa {
2     private String nome;
3     private int cpf;
4
5     Pessoa(String nome, int cpf) {
6         this.nome = nome;
7         this.cpf = cpf;
8     }
9     public String getNome() {
10         return nome;
11     }
12     public int getCpf() {
13         return cpf;
14     }
15     public String toString() {
16         return "\nNome: " + nome + " CPF: " + cpf;
17     }
18 }
```


Empregado

```
1 public class Empregado extends Pessoa {
2     private float salario;
3
4     public Empregado(String nome, int cpf, float salario) {
5         super(nome, cpf);
6         this.salario = salario;
7     }
8
9     public float getSalario() {
10         return salario;
11     }
12
13     public String toString() {
14         String resultado;
15         resultado = super.toString() + "\n";
16         resultado += "Salario: R$ " + salario + " reais\n\n";
17         return resultado;
18     }
19 }
```

Chefe de Departamento

```
1 public class ChefeDepartamento extends Empregado {
2     private String departamento;
3     public ChefeDepartamento(String nome, int cpf, float salario, String departamento) {
4         super(nome, cpf, salario);
5         this.departamento = departamento;
6     }
7
8     public String getDepartamento() {
9         return departamento;
10    }
11
12    public String toString() {
13        String resultado;
14        resultado = super.toString() + "\n";
15        resultado += "Departamento: " + departamento + "\n";
16        return resultado;
17    }
18 }
```

```
1 class Emprestimo {
2     public static void main(String[] args) {
3         Pessoa pessoa = new Pessoa("lucilia", 123);
4         Empregado func1 = new Empregado("bruna", 456, 10000f);
5         Empregado func2 = new Empregado("andre", 789, 15000f);
6         Empregado func3 = new Empregado("fulano", 777, 1251.3f);
7         ChefeDepartamento chefe = new ChefeDepartamento("boss", 654, 25.25f, "CPD");
8         System.out.print(pessoa);
9         System.out.println("\nValor do emprestimo: R$ " + calculaEmprestimo(pessoa));
10        System.out.print(func1);
11        System.out.println("\nValor do emprestimo: R$ " + calculaEmprestimo(func1));
12        System.out.print(func2);
13        System.out.println("\nValor do emprestimo: R$ " + calculaEmprestimo(func2));
14        System.out.print(func3);
15        System.out.println("\nValor do emprestimo: R$ " + calculaEmprestimo(func3));
16        System.out.print(chefe);
17        System.out.println("\nValor do emprestimo: R$ " + calculaEmprestimo(chefe));
18    }
19
20    public static float calculaEmprestimo(Pessoa pessoa) {
21        return 1000f;
22    }
23
24    public static float calculaEmprestimo(Empregado funcionario) {
25        float emprestimo = 0f;
26        if (funcionario instanceof ChefeDepartamento) {
27            emprestimo = 4.0f * funcionario.getSalario();
28        } else {
29            if (funcionario instanceof Empregado) {
30                emprestimo = 2.0f * funcionario.getSalario();
31            }
32        }
33        return emprestimo;
34    }
35 }
```


Dúvidas?

